



UD7_Práctica1 - Procedementos.

Índice

UD7_Práctica1 - Procedementos.....	1
Documentación e información de entrega.....	2
O que hai que entregar.....	2
Procedimentos.....	3
Exercicios sobre Procedimentos.....	3



Documentación e información de entrega.

A base de datos de jardinería téndela dispoñible no curso da aula virtual, a carón do bloque de exercicios.

Moitos dos exercicios fanse sobre as mesmas BD, polo que podedes aproveitar o que teñades feito duns exercicios para outros.

O que hai que entregar.

Un documento PDF que inclúa as imaxes seguintes:

- **O código de cada exercicio.** En caso de que o código non quepa nunha imaxe:
 - Copia o código no documento.
 - Inclúe unha imaxe tamén onde se vexa a rutina incluída na BD. Debe verse o nome e a cabeceira da rutina.
- **Os casos de proba** de cada exercicio. Isto inclúe unha imaxe por cada posible resultado da rutina.

As imaxes deben ser válidas: *incluír o teu nome nalgues, o comando, o resultado da execución e o Action Output*



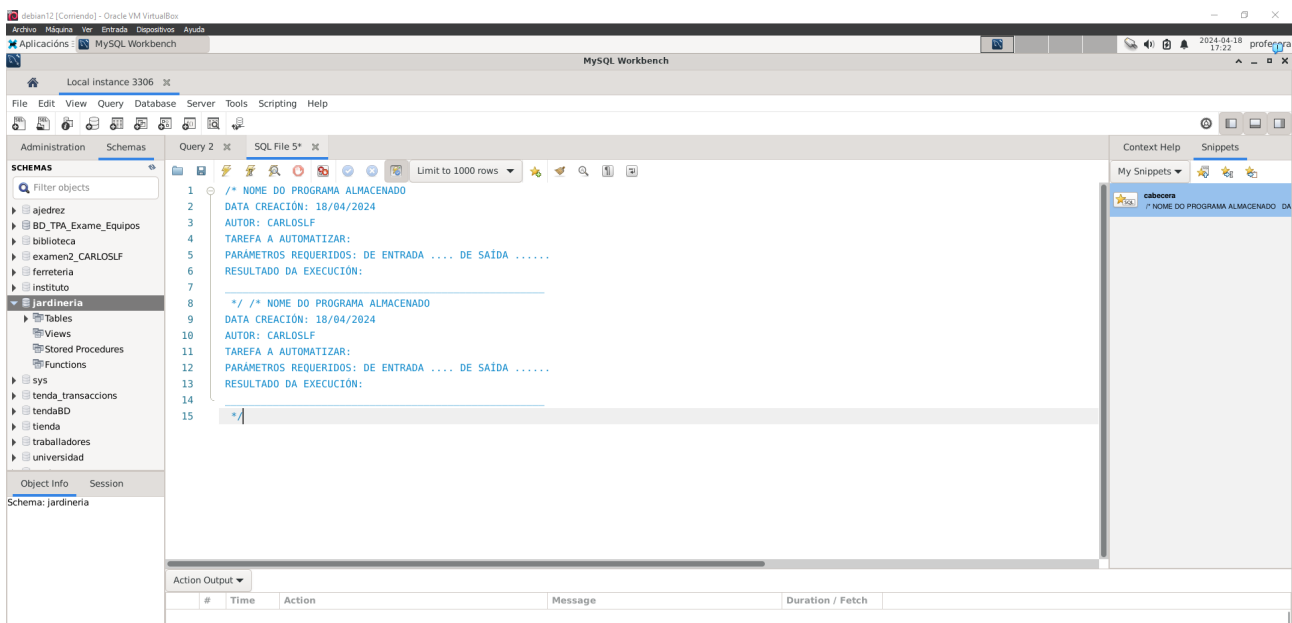
Procedimentos.

Exercicios sobre Procedimentos.

1. Crear un snippets como o do modelo que inclúa o teu nome no campo

'AUTOR'. Engadirase a cada unha das funcións da práctica.

```
1 /*
2 NOME DO PROGRAMA ALMACENADO
3 DATA CREACIÓN:
4 AUTOR:
5 TAREFA A AUTOMATIZAR:
6 PARÁMETROS REQUERIDOS: DE ENTRADA .... DE SAÍDA .....
7 RESULTADO DA EXECUCIÓN:
8 _____
9 */
10 Código procedimento / función / trigger
```



2. Escribe un procedemento que reciba como parámetro de entrada unha forma de pago, que será unha cadea de caracteres (Exemplo: PayPal, Transferencia, etc). E devolva como saída o pago de máximo valor realizado para esa forma de pago. Deberá facer uso da táboa pago da base de datos jardinería.

Os **requerimentos técnicos** serán os seguintes:

- Empregaremos transaccións.
- Empregaremos sentenzas preparadas.



- Empregaremos un handler de excepcións para excepcións e warnings. Ao empregar sentenzas preparadas, o handler non só revertirá as operacións incluídas na transacción, senón que terá que liberar a memoria da sentenza preparada.

```
DELIMITER ;;
/* NOME DO PROGRAMA ALMACENADO
DATA CREACIÓN: 18/04/2024
AUTOR: CARLOS LF
TAREFA A AUTOMATIZAR:
PARAMETROS REQUERIDOS: DE ENTRADA .... DE SAIDA .....
RESULTADO DA EXECUCIÓN:
*/
CREATE DEFINER='root'@'localhost' PROCEDURE 'ObtenerMaximoPagoPorForma' (
  in p_forma_pago VARCHAR(40), out p_maximo_pago DECIMAL(15,2),
  out p_error_message VARCHAR(255))
BEGIN
  DECLARE v_maximo_pago DECIMAL(15,2);
  DECLARE f_pago VARCHAR(40);

  DECLARE v_tipo_error INT DEFAULT 0;
  DECLARE error_per CONDITION FOR SQLSTATE '45000';
  DECLARE EXIT HANDLER FOR error_per
  BEGIN
    ROLLBACK;
    CASE v_tipo_error
    WHEN 1 THEN SET p_error_message = 'Forma de pago no valida';
    WHEN 2 THEN SET p_error_message = 'Forma de pago sin valor';
    END CASE;
END;

Action Output
# Time Action Message Duration / Fetch
11 21:03:45 Apply changes to PagoMaximo No changes detected
12 21:07:15 ... CARLOS LF CREATE DEFINER='root'@'localhost' PROC... 0 rows(s) affected 0.0096 sec
```

```
WHEN 2 THEN SET p_error_message = 'Forma de pago sin valor';
END CASE;

END;
DECLARE EXIT HANDLER FOR SQLEXCEPTION, SQLWARNING
BEGIN
  -- Revertimos la transacción si hay una excepción o warning
  SET p_error_message = 'Error inesperado';
  ROLLBACK;
END;
-- Iniciamos la transacción
START TRANSACTION;

IF NOT EXISTS (SELECT 1 FROM pago WHERE forma_pago = p_forma_pago) THEN
  SET v_tipo_error = 1;
  SIGNAL error_per;
END IF;
-- Preparamos la sentencia SQL
SET @f_pago = p_forma_pago;
PREPARE preparada FROM 'SELECT MAX(total) FROM pago WHERE forma_pago LIKE ? INTO @v_maximo_pago';

-- Ejecutamos la sentencia preparada
EXECUTE preparada USING @f_pago;

Action Output
# Time Action Message Duration / Fetch
11 21:03:45 Apply changes to PagoMaximo No changes detected
12 21:07:15 ... CARLOS LF CREATE DEFINER='root'@'localhost' PROC... 0 rows(s) affected 0.0096 sec
```



The screenshot shows the MySQL Workbench interface with a SQL query being executed. The query is a stored procedure that updates the 'PagoMaximo' routine. It includes comments in Spanish explaining the steps: checking for existence, preparing the statement, executing it, and then committing the changes. The 'Action Output' pane at the bottom shows the execution results.

```
39
40 IF NOT EXISTS (SELECT 1 FROM pago WHERE forma_pago = p_forma_pago) THEN
41 SET v_tipo_error = 1;
42 SIGNAL error_per;
43 END IF;
44 -- Preparamos la sentencia SQL
45 SET @f_pago = p_forma_pago;
46
47 PREPARE preparada FROM 'SELECT MAX(total) FROM pago WHERE forma_pago LIKE ? INTO @v_maximo_pago';
48
49 -- Ejecutamos la sentencia preparada
50 EXECUTE preparada USING @f_pago;
51
52
53
54 -- Devolvemos el resultado
55 IF @v_maximo_pago IS NOT NULL THEN
56 SET p_maximo_pago = @v_maximo_pago;
57 COMMIT;
58 ELSE
59 SET v_tipo_error = 2;
60 SIGNAL error_per;
61 END IF;
62
63 -- Liberamos la memoria de la sentencia preparada
64 DEALLOCATE PREPARE preparada;
65 END ;;
66 DELIMITER ;
```

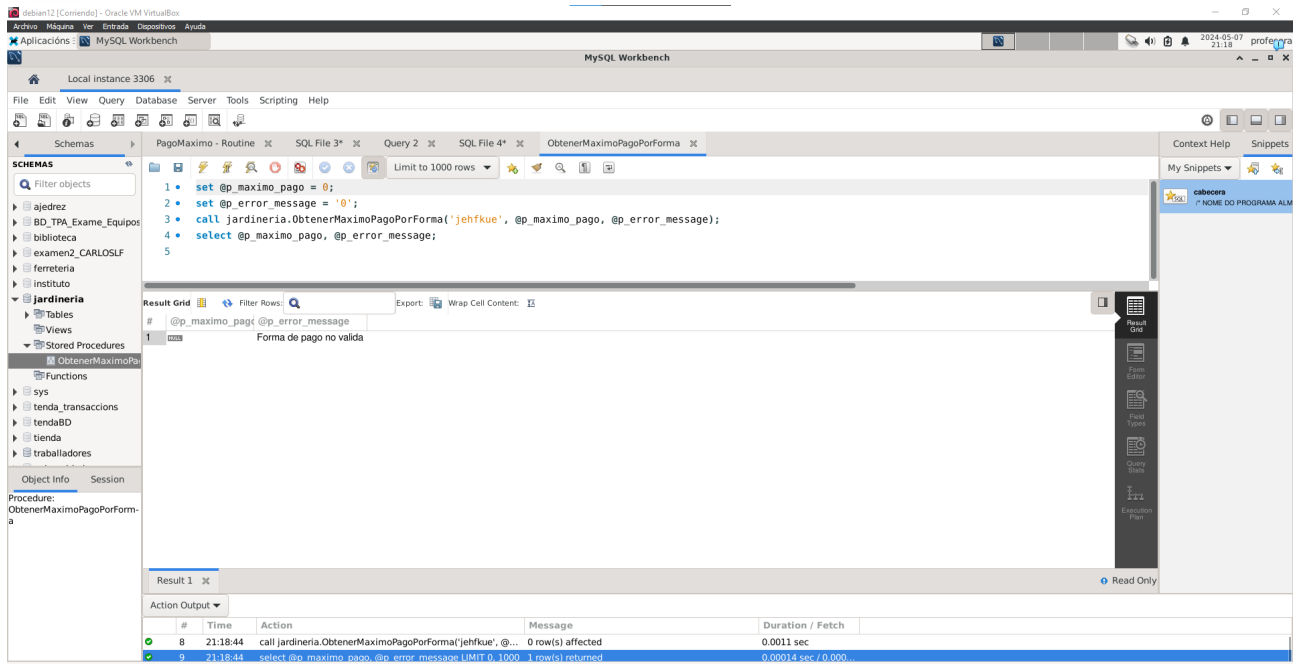
#	Time	Action	Message	Duration / Fetch
11	21:03:45	Apply changes to PagoMaximo	No changes detected	
12	21:07:15	-- CARLOS LF CREATE DEFINER=root@localhost PROC...	0 row(s) affected	0.0096 sec

The screenshot shows the MySQL Workbench interface with a SQL query being executed. The query is a stored procedure that updates the 'ObtenerMaximoPagoPorForma' routine. It includes comments in Spanish explaining the steps: setting variables, calling the routine, and then selecting the results. The 'Result Grid' pane shows the execution results.

```
1 • set @p_maximo_pago = 0;
2 • set @p_error_message = '0';
3 • call jardineria.ObtenerMaximoPagoPorForma('PayPal', @p_maximo_pago, @p_error_message);
4 • select @p_maximo_pago, @p_error_message;
```

#	@p_maximo_pago	@p_error_message
1	20000.00	

#	Time	Action	Message	Duration / Fetch
4	21:17:46	call jardineria.ObtenerMaximoPagoPorForma('PayPal', @p...	0 row(s) affected	0.0031 sec
5	21:17:46	select @p_maximo_pago, @p_error_message LIMIT 0, 1000	1 row(s) returned	0.00038 sec / 0.000...



3. Crea un novo procedemento que amplía o exercicio anterior (podes facer unha copia do anterior e comezar dende ese punto). O **requerimentos técnicos** e documentación serán os mesmos que no exercicio anterior.

O novo procedemento recibe como parámetro de entrada unha forma de pago, igual que no anterior, pero vai devolver como saída os seguintes valores tendo en conta a forma de pago seleccionada como parámetro de entrada:

- o pago de máximo valor,
- o pago de mínimo valor,
- o valor medio dos pagos realizados,
- a suma de todos os pagos,
- o número de pagos realizados para esa forma de pago.



```
1  /* NOME DO PROGRAMA ALMACENADO
2  DATA CREACIÓN: 18/04/2024
3  AUTOR: CARLOS LF
4  TAREFA A AUTOMATIZAR:
5  PARÁMETROS REQUERIDOS: DE ENTRADA .... DE SAÍDA .....
6  RESULTADO DA EXECUCIÓN:
7
8  */
9
10 DELIMITER ;;
11 CREATE DEFINER='root'@'localhost' PROCEDURE `ObtenerMaximoPagoPorForma`()
12
13   IN p_forma_pago VARCHAR(40),
14   OUT p_maximo_pago DECIMAL(15,2),
15   OUT p_minimo_pago DECIMAL(15,2),
16   OUT p_valor_medio DECIMAL(15,2),
17   OUT p_suma_total DECIMAL(15,2),
18   OUT p_numero_pagos INT,
19   OUT p_error_message VARCHAR(255)
20 BEGIN
21   -- Declaramos variables
22   DECLARE v_maximo_pago DECIMAL(15,2);
23   DECLARE v_minimo_pago DECIMAL(15,2);
24   DECLARE v_valor_medio DECIMAL(15,2);
25   DECLARE v_suma_total DECIMAL(15,2);
26   DECLARE v_numero_pagos INT;
27
28   DECLARE f_pago VARCHAR(40);
29   DECLARE v_tipo_error INT DEFAULT 0;
30   DECLARE error_per CONDITION FOR SQLSTATE '45000';
31   DECLARE EXIT HANDLER FOR error_per
32   BEGIN
33     ROLLBACK;
```

#	Time	Action	Message	Duration / Fetch
1	21:19:23	CREATE DEFINER='root'@'localhost' PROCEDURE `Obte...	0 row(s) affected	0.011 sec

```
33   ROLLBACK;
34   CASE v_tipo_error
35   WHEN 1 THEN SET p_error_message = 'Forma de pago no válida';
36   WHEN 2 THEN SET p_error_message = 'Forma de pago sin valor';
37   END CASE;
38   END;
39   DECLARE EXIT HANDLER FOR SQLWARNING, SQLWARNING
40   BEGIN
41     -- Revertimos la transacción si hay una excepción o warning
42     SET p_error_message = 'Error inesperado';
43     ROLLBACK;
44   END;
45   -- Iniciamos la transacción
46   START TRANSACTION;
47   IF NOT EXISTS (SELECT 1 FROM pago WHERE forma_pago = p_forma_pago) THEN
48     SET v_tipo_error = 1;
49     SIGNAL error_per;
50   END IF;
51   -- Obtener el pago máximo
52   SET @f_pago = p_forma_pago;
53   PREPARE preparada_maximo_pago FROM 'SELECT MAX(total) FROM pago WHERE forma_pago LIKE ? INTO @v_maximo_pago';
54   -- Ejecutamos la sentencia preparada
55   EXECUTE preparada_maximo_pago USING @f_pago;
56   DEALLOCATE PREPARE preparada_maximo_pago;
57
58   -- Obtener el pago mínimo
59   PREPARE preparada_minimo_pago FROM 'SELECT MIN(total) FROM pago WHERE forma_pago LIKE ? INTO @v_minimo_pago';
60   EXECUTE preparada_minimo_pago USING @f_pago;
61   DEALLOCATE PREPARE preparada_minimo_pago;
62
63   -- Obtener el valor medio de los pagos
```

#	Time	Action	Message	Duration / Fetch
1	21:19:23	CREATE DEFINER='root'@'localhost' PROCEDURE `Obte...	0 row(s) affected	0.011 sec



MySQL Workbench

Local Instance 3306

File Edit View Query Database Server Tools Scripting Help

Schemas

Filter objects

ajedrez

BD_TPA_Exame_Equipos

biblioteca

examen2_CARLOSIF

ferreteria

instituto

jardineria

Tables

Views

Stored Procedures

ObtenerMaximoPagoPorFormaEJ3

Functions

sys

tenda_transaccions

tendaBD

tienda

trabajadores

Object Info Session

Procedure: ObtenerMaximoPagoPorFormaEJ3

ObtenerMaximoPagoPorFormaEJ3

```
63 -- Obtener el valor medio de los pagos
64 PREPARE preparada_valor_medio FROM 'SELECT AVG(total) FROM pago WHERE forma_pago = ? INTO @v_valor_medio';
65 EXECUTE preparada_valor_medio USING @f_pago;
66 DEALLOCATE PREPARE preparada_valor_medio;
67
68 -- Obtener la suma total de los pagos
69 PREPARE preparada_suma_total FROM 'SELECT SUM(total) FROM pago WHERE forma_pago = ? INTO @v_suma_total';
70 EXECUTE preparada_suma_total USING @f_pago;
71 DEALLOCATE PREPARE preparada_suma_total;
72
73 -- Obtener el número de pagos
74 PREPARE preparada_numero_pagos FROM 'SELECT COUNT(*) FROM pago WHERE forma_pago = ? INTO @v_numero_pagos';
75 EXECUTE preparada_numero_pagos USING @f_pago;
76 DEALLOCATE PREPARE preparada_numero_pagos;
77
78 -- Asignar los valores de salida
79 SELECT @v_maximo_pago INTO p_maximo_pago;
80 SELECT @v_minimo_pago INTO p_minimo_pago;
81 SELECT @v_valor_medio INTO p_valor_medio;
82 SELECT @v_suma_total INTO p_suma_total;
83 SELECT @v_numero_pagos INTO p_numero_pagos;
84
85 IF @v_maximo_pago IS NOT NULL THEN
86 COMMIT;
87 ELSE
88 SET v_tipo_error = 2;
89 SIGNAL error_per;
90 END IF;
91
92 -- Liberamos la memoria de la sentencia preparada
93 DEALLOCATE PREPARE preparada_maximo_pago;
94
95 END ;;
96 DELIMITER ;
```

Action Output

#	Time	Action	Message	Duration / Fetch
1	21:19:23	CREATE DEFINER='root'@'localhost' PROCEDURE Obte...	0 row(s) affected	0.011 sec

MySQL Workbench

Local Instance 3306

File Edit View Query Database Server Tools Scripting Help

Schemas

Filter objects

ajedrez

BD_TPA_Exame_Equipos

biblioteca

examen2_CARLOSIF

ferreteria

instituto

jardineria

Tables

Views

Stored Procedures

ObtenerMaximoPagoPorFormaEJ3

Functions

sys

tenda_transaccions

tendaBD

tienda

trabajadores

Object Info Session

Procedure: ObtenerMaximoPagoPorFormaEJ3

ObtenerMaximoPagoPorFormaEJ3

```
1 set @p_maximo_pago = 0;
2 set @p_minimo_pago = 0;
3 set @p_valor_medio = 0;
4 set @p_suma_total = 0;
5 set @p_numero_pagos = 0;
6 set @p_error_message = '0';
```

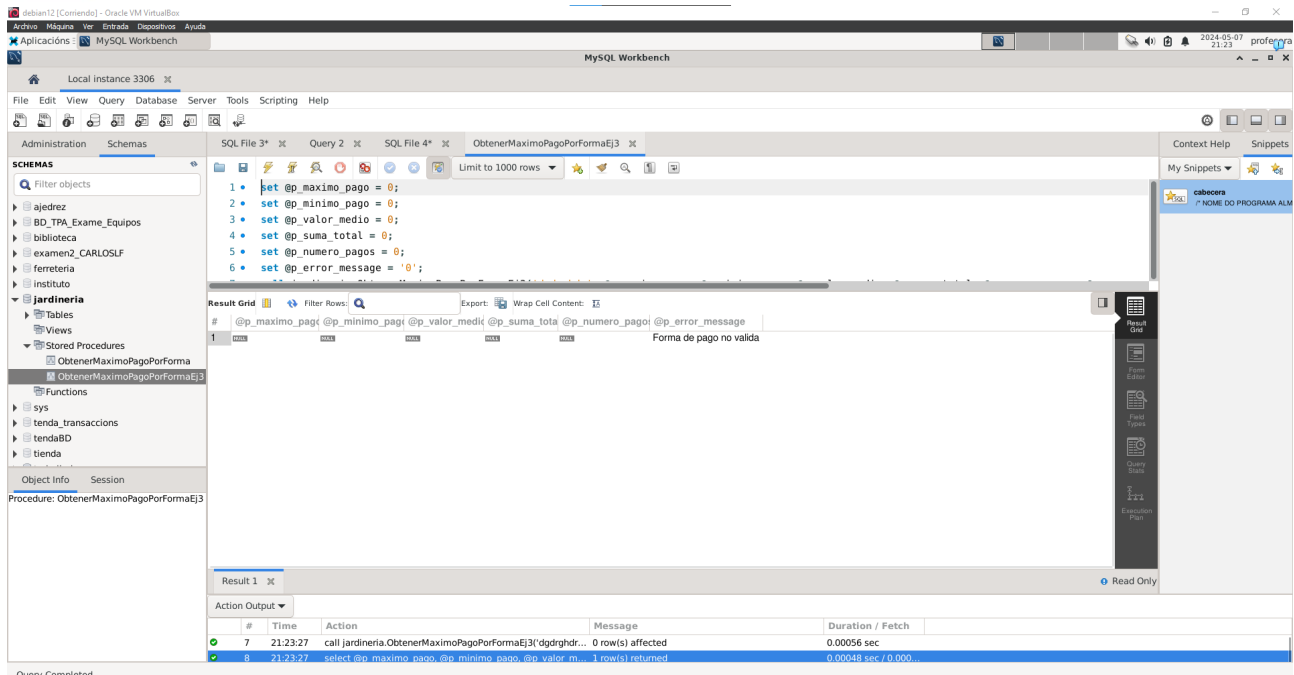
Result Grid

#	@p_maximo_pago	@p_minimo_pago	@p_valor_medio	@p_suma_total	@p_numero_pagos	@p_error_message
1	20000.00	232.00	7156.50	171756.00	24	Error inesperado

Action Output

#	Time	Action	Message	Duration / Fetch
7	21:23:02	call jardineria.ObtenerMaximoPagoPorFormaEJ3('PayPal', ...	0 row(s) affected	0.0024 sec
8	21:23:02	select @p_maximo_pago, @p_minimo_pago, @p_valor_m...	1 row(s) returned	0.0012 sec / 0.0000

Query Completed



4. Procedimiento: calcular_pagos_pendientes

- Descripción: Deberá calcular os pagos pendentes de todos os clientes. Para saber se un cliente ten algún pago pendiente deberemos calcular cal é a cantidade de todos os pedidos e os pagos realizados. Se a cantidade dos pedidos é maior que a dos pagos entón ese cliente ten pagos pendentes.
- Os **requerimentos técnicos** serán os mesmos que nos exercicios anteriores.
- Deberá insertar nunha táboa chamada clientes_con_pagos_pendientes os seguintes datos:
 - id_cliente
 - suma_total_pedidos
 - suma_total_pagos
 - pendiente_de_pago



MySQL Workbench interface showing the creation of a stored procedure. The SQL editor contains the following code:

```
1 /* NOME DO PROGRAMA ALMACENADO
2 DATA CREACIÓN: 18/04/2024
3 AUTOR: CARLOS LF
4 TAREFA A AUTOMATIZAR:
5 PARAMETROS REQUERIDOS: DE ENTRADA .... DE SAIDA .....
6 RESULTADO DA EXECUCIÓN:
7
8 */
9
10 DELIMITER ;;
11 CREATE DEFINER='root'@'localhost' PROCEDURE `calcular_pagos_pendientes`()
12 BEGIN
13 -- Hector Garaboa
14 DECLARE EXIT HANDLER FOR SOLEXCEPTION, SQLWARNING
15 BEGIN
16 -- Si ocurre una excepción o warning, reversionamos la transacción y liberamos la sentencia preparada
17 ROLLBACK;
18 DEALLOCATE PREPARE preparada;
19 END;
20
21 -- Iniciamos la transacción
22 START TRANSACTION;
23
24 -- Preparamos la sentencia
25 SET @sql = CONCAT('INSERT INTO clientes_con_pagos_pendientes (id_cliente, suma_total_pedidos, suma_total_pagos, pendiente_de_pago) ',
```

The Action Output table shows the execution result:

#	Time	Action	Message	Duration / Fetch
1	21:24:47	CREATE DEFINER='root'@'localhost' PROCEDURE `calcu...	0 row(s) affected	0.0098 sec

MySQL Workbench interface showing the continuation of the stored procedure code. The SQL editor contains the following code:

```
18 DEALLOCATE PREPARE preparada;
19 END;
20
21 -- Iniciamos la transacción
22 START TRANSACTION;
23
24 -- Preparamos la sentencia
25 SET @sql = CONCAT('INSERT INTO clientes_con_pagos_pendientes (id_cliente, suma_total_pedidos, suma_total_pagos, pendiente_de_pago) ',
26 'SELECT c.codigo_cliente, ',
27 'IFNULL((SELECT SUM(dp.cantidad * dp.precio_unidad) FROM detalle_pedido dp INNER JOIN pedido p ON dp.codigo_pedido = p.codigo_pedido) AS suma_total_pedidos, ',
28 'IFNULL((SELECT SUM(total) FROM pago WHERE codigo_cliente = c.codigo_cliente), 0) AS suma_total_pagos, ',
29 'IFNULL((SELECT SUM(dp.cantidad * dp.precio_unidad) FROM detalle_pedido dp INNER JOIN pedido p ON dp.codigo_pedido = p.codigo_pedido) AS suma_total_pagos, ',
30 'FROM cliente c ',
31 'HAVING suma_total_pedidos > suma_total_pagos');
32
33 PREPARE preparada FROM @sql;
34 EXECUTE preparada;
35
36 -- Liberamos la sentencia preparada
37 DEALLOCATE PREPARE preparada;
38
39 -- Si todo ha ido bien, hacemos commit de la transacción
40 COMMIT;
41 END ;;
42 DELIMITER ;
```

The Action Output table shows the execution result:

#	Time	Action	Message	Duration / Fetch
1	21:24:47	CREATE DEFINER='root'@'localhost' PROCEDURE `calcu...	0 row(s) affected	0.0098 sec

MySQL Workbench interface showing the creation of a table. The SQL editor contains the following code:

```
1 DROP TABLE IF EXISTS `clientes_con_pagos_pendientes`;
2 /*!40101 SET @saved_cs_client = @@character_set_client */;
3 /*!50503 SET character_set_client = utf8mb4 */;
4 CREATE TABLE `clientes_con_pagos_pendientes` (
5 `id_cliente` int NOT NULL,
6 `suma_total_pedidos` decimal(15,2) DEFAULT NULL,
7 `suma_total_pagos` decimal(15,2) DEFAULT NULL,
8 `pendiente_de_pago` decimal(15,2) DEFAULT NULL,
9 PRIMARY KEY (`id_cliente`)
10 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

The Action Output table shows the execution result:

#	Time	Action	Message	Duration / Fetch
3	21:29:41	/*!50503 SET character_set_client = utf8mb4 */	0 row(s) affected	0.00023 sec
4	21:29:41	CREATE TABLE `clientes_con_pagos_pendientes` (`id_cli...	0 row(s) affected	0.024 sec



MySQL Workbench interface showing a query execution. The query is:

```
CALL jardineria.calcular_pagos_pendientes();
```

The Action Output shows:

#	Time	Action	Message	Duration / Fetch
1	21:30:45	call jardineria.calcular_pagos_pendientes()	0 row(s) affected	0.038 sec

MySQL Workbench interface showing a query execution. The query is:

```
SELECT * FROM jardineria.clientes_con_pagos_pendientes;
```

The Result Grid shows the following data:

#	id_cliente	suma_total_pedido	suma_total_pago	pendiente_de_pago
1	1	10365.00	4000.00	6365.00
2	3	13726.00	10926.00	2800.00
3	16	7899.00	4399.00	3500.00
4	26	18979.00	18846.00	133.00
5	30	11363.00	7863.00	3500.00
6	36	3500.00	0.00	3500.00

The Action Output shows:

#	Time	Action	Message	Duration / Fetch
1	21:31:11	SELECT * FROM jardineria.clientes_con_pagos_pendientes ...	6 row(s) returned	0.00038 sec / 0.000...